HTTP Messages, Caching and Electronic Mail

Lecture 05

Computer Networks and Distributed Programming

Where we left the Web

We know what HTTP does and what it costs, and nothing yet about what it says.

- A page is a set of objects, each with a URL, each fetched with a request and returned in a response.
- The exchange runs over TCP, and the protocol pays for reliability in round trips, not in code of its own.
- HTTP is stateless: the server keeps nothing of one exchange for the next one.

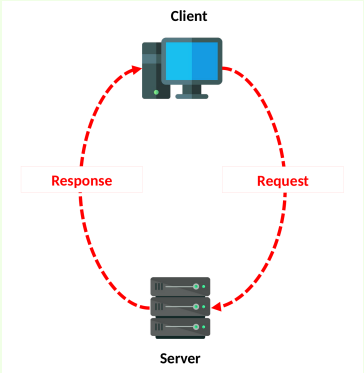
Today we open the two messages. Everything else in this lecture — shopping carts, caches, proxies — turns out to be one header line.

Two message formats, and only two

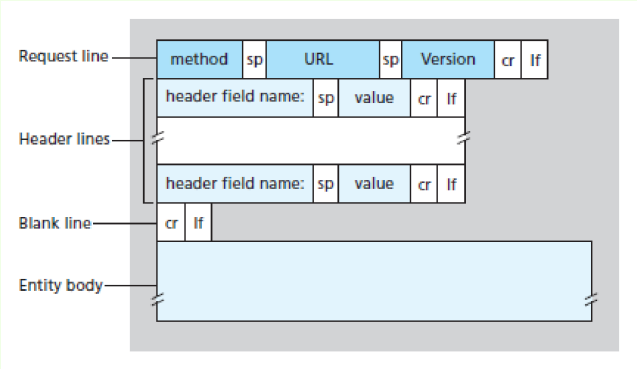
- Request: names the command the server has to carry out: send me this page, take this form.
- Response: reports the outcome of that command, and usually carries the object with it.

Both are written in ordinary ASCII text, so a human can read them off the wire without a decoder.

That is unusual, and it is why the rest of this lecture can be done by reading.



The request message



- A request line — method, URL, version — then any number of header lines, one parameter each.
- A blank line closes them; after it, the optional entity body.

The separators are literal characters: `sp` for space, `cr` and `lf` for the end of line.

A request, line by line

A browser asking `www.someschool.edu` for one page. Five lines, and the body is empty.

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

The first line is the request line: method `GET`, path `/somedir/page.html`, version `HTTP/1.1`. The other four are headers.

What the request headers say

```
Host: www.someschool.edu      Connection: close
User-agent: Mozilla/5.0       Accept-language: fr
```

- Host: the host the object resides on — which need not be the machine the connection was opened to, if a proxy sits in between.
- Connection: `close` asks the server to hang up after this one response: a non-persistent exchange, by request.
- User-agent: the browser type, so that the server may return a different version of the same URL.
- Accept-language: the user would rather have the French copy, if the server has one.

Custom headers are allowed: the list is open, and each side ignores what it does not understand.

The methods

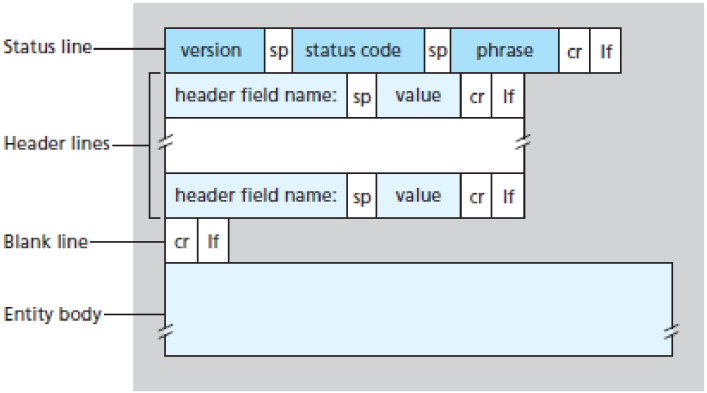
Method	What it asks	Body
`GET`	send me this object	empty
`POST`	take these values and act on them	the form fields
`HEAD`	send the headers, not the object	empty
`PUT`	store this object at this path	the object
`DELETE`	remove this object	empty

- The great majority of requests are `GET`: every object of every page you open.
- `HEAD` is a debugging tool — ask what the server would send, without paying for it.

A form always uses POST, then?

No: many use `GET` and put the fields in the query part of the URL.

The response message



Same shape, with the request line replaced by a status line: the version the server speaks, a numeric status code, and the phrase that spells it out for a human.

Five classes of status code

Range	Class	Meaning
100–199	informational	about the request, which is still in progress
200–299	successful	the command was carried out
300–399	redirection	something more is required of the client
400–499	client error	the request was wrong
500–599	server error	the request was fine, the server was not

The first digit is the whole story: a client that understands nothing else can still tell success from failure.

Five you will actually meet

`200 OK`	the object is in the body below
`301 Moved Permanently`	the new URL is in the `Location:` header
`400 Bad Request`	the server could not make sense of the request
`404 Not Found`	no such object on this server
`505 HTTP Version Not Supported`	the server does not speak that version

A sixth, `304 Not Modified`, exists only to make caching work: we come back to it later today.

A response, line by line

The answer to the request two slides back.

```
HTTP/1.1 200 OK
Connection: close
Date: Tue, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html

(data data data data data ...)
```

Status line, six header lines, a blank line, and then the object itself.

What the response headers say

- Connection: `close`: the server will hang up once this message is out.
- Date: when the response was built and sent — not when the object was written.
- Server: which program answered, `Apache/2.2.3` here: the mirror image of `User-agent`.
- Last-Modified: when the object was last changed. Remember this one: caching is built on it.
- Content-Length: how many bytes of object follow the blank line.
- Content-Type: what those bytes are. The type is settled here, not by the file extension.

Talking HTTP by hand

Nothing in a request is generated by magic: you can type one yourself. `-C` is not optional: HTTP ends every line with `CRLF`, and `nc` alone sends only `LF` — try it, and this server answers 400 Bad Request.

```
$ nc -C info.cern.ch 80          opens a TCP connection to port 80
GET /hypertext/WWW/TheProject.html HTTP/1.1
Host: info.cern.ch
                                 the blank line ends the request
HTTP/1.1 200 OK
...
```

The server answers a program that is not a browser, because it cannot tell the difference: it only ever sees those bytes.

Replace `GET` with `HEAD` and you get the headers alone — the same metadata, none of the traffic.

Statelessness has a price

Statelessness is what lets one machine serve thousands of connections at once. But it makes a whole class of services impossible.

- A shopping cart is what the same user put in it over several requests — and the server remembers none of them.
- Recommendations, a session that stays logged in, a language you chose once: all of them are per client.
- So the identity has to be carried in the messages, since it cannot live in the server's memory of you.

Cookie

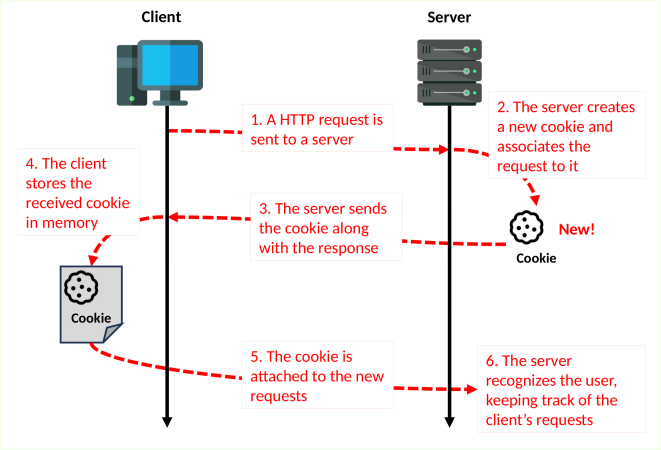
A short alphanumeric identifier that a server hands to a client, and that the client hands back on every later request.

Four pieces make a cookie work

1. A `Set-cookie:` header line in the HTTP response, where the server hands out the identifier.
2. A `Cookie:` header line in every later HTTP request, where the client hands it back.
3. A file on the client machine, one line per site, kept by the browser.
4. A back-end database on the server, indexed by that identifier.

Two header lines and two stores. Note where the state actually is: at the two ends, never in the protocol.

The mechanism, step by step

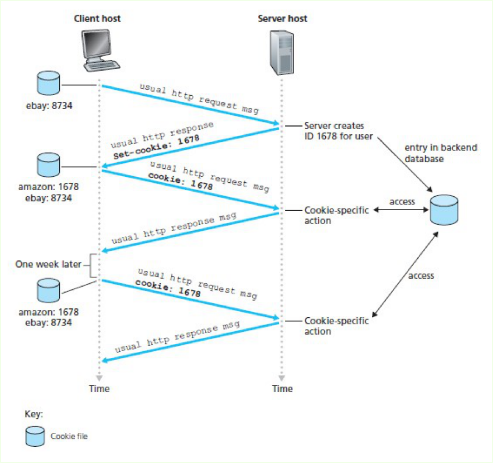


The cookie is created by the server, stored by the client, and returned by the client from then on.

The cookie file, one week later

A client that uses eBay regularly reaches Amazon for the first time.

- The server invents 1678 and returns `Set-cookie: 1678`.
- The browser appends host and number to its cookie file.
- Every later request carries `Cookie: 1678`: the server looks the user up.
- A week later nothing changes: the same line goes out again.



What cookies buy, and what they cost

- Cart: the list of intended purchases survives across requests, and is paid for once at the end.
- Registration: name, address and card sit in the database beside the number: one-click shopping is this, and nothing more.
- Recommendations: the server knows which pages user 1678 opened, in what order, and at what times.

It never needed to know the name to do any of that. Give it once — at registration — and the whole history acquires one.

Hence the controversy: the same mechanism that carries the cart is a tracking record, and it can be sold.

If the cookie file is deleted

You clear your browser data mid-purchase. What happens to the cart?

The next request arrives without the `Cookie:` line. The server has no way to tell it from a first visit, so it issues a new identifier: the cart is empty, and the old one is still in the database, now unreachable.

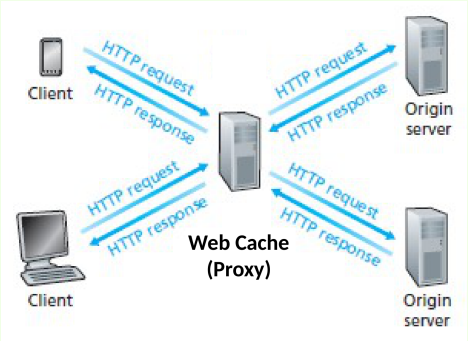
- The server was never keeping your session: it was keeping a row, and you were carrying the key to it.
- Which is also why the same account looks different on your phone: a different file, a different key.

HTTP has not become stateful. The state was built on top of it, out of two header lines.

The Web cache

Web cache, or proxy server

A network entity that satisfies HTTP requests on behalf of an origin server, out of its own disk.



A browser can be configured so that every request goes there first.

A request through a cache

Asking for `http://www.someschool.edu/campus.gif` through a cache:

1. The browser opens a TCP connection to the cache and sends the request there.
2. The cache looks for the object on its own disk. If it has it, it answers, and it ends here.
3. If it does not, the cache opens a connection to the origin server and asks for the object.
4. It stores the copy, then forwards it to the browser on the connection already open.

So a cache is both: a server towards the browser, a client towards the origin server. Nothing in the protocol had to change.

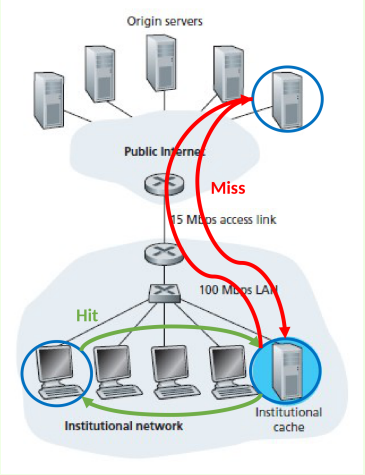
Hit and miss

Hit

A request the cache satisfies without contacting the origin server.

- A hit stays on the 100 Mbps LAN; a miss crosses the 15 Mbps access link twice.
- Hit rates in practice run from 0.2 to 0.7, and grow with the number of clients sharing the cache.

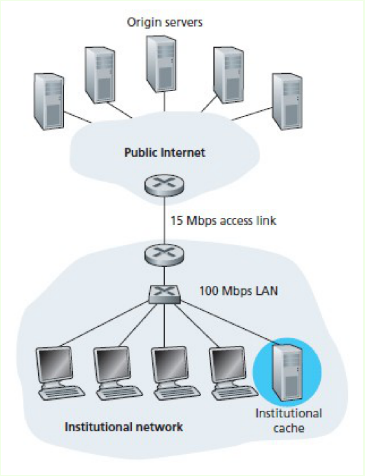
Up to seven requests in ten never leave the building.



Why an institution installs one

- Response time: the cache is one hop away on a fast LAN, and the origin server is on the other side of an ocean.
- Traffic: what the cache serves never crosses the access link, so the institution buys less bandwidth and buys it later.

The second reason is the one that pays for the machine. A university, a company, a residential ISP: all install them for the link, not for the users.



What the cache is worth

The figure on the previous slide, with numbers

Average object 1 Mbit, 15 requests per second from the institution, and two seconds on average to fetch anything from the far side of the Internet.

- Demand on the access link: $15 \times 1 = 15$ Mbps, on a link of exactly 15 Mbps.
- A link served at the whole of its capacity does not keep up: the queue in front of it grows without bound, and the wait becomes minutes.
- Upgrading the link to 100 Mbps fixes it, and costs money every month.
- A cache with hit rate 0.4 leaves only 60% of the requests to cross: demand falls to 9 Mbps, and the average wait to about 1.2 seconds.

Cheaper than the upgrade, and faster than it. Why the queue behaves that way is a question for the transport layer.

The copy may be stale

Caching buys time and introduces a problem: the object in the cache may have been modified at the origin server since the day it was stored.

Conditional GET

A request that uses the `GET` method and carries an `If-modified-since:` header line.

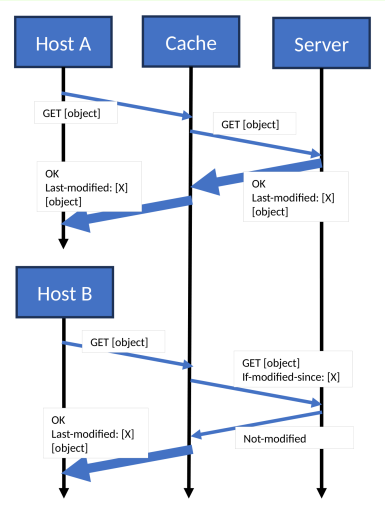
- The cache kept the `Last-Modified:` value it was given, and now sends it back: only if it has changed since then.
- Unchanged, the server answers `304 Not Modified` with an empty body, and the cache serves its own copy.

A check costs two small messages instead of the whole object.

The conditional GET

- Host A asks first: the cache has nothing, fetches from the server, keeps the object and its date.
- Host B asks later. The cache does not forward the request: it asks the server whether the copy is still good.
- The object crosses the network once, and is served twice.

Nothing here is a new protocol: it is two header lines and one status code.



The same exchange, in headers

What the cache stored, one week earlier:

```
HTTP/1.1 200 OK
Last-Modified: Wed, 9 Sep 2015 09:23:24      kept with the object
```

What it sends today, and what comes back:

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
If-modified-since: Wed, 9 Sep 2015 09:23:24 the same value, returned

HTTP/1.1 304 Not Modified
Date: Sat, 10 Oct 2015 15:39:29
                                            empty entity body
```

The image was never sent. That is the entire saving, and it is why `Last-Modified:` is on every response.

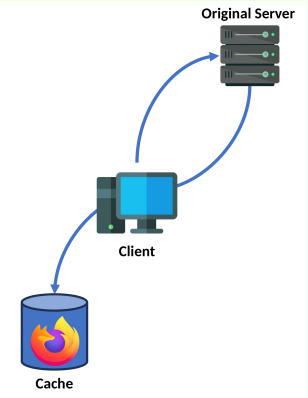
The browser caches too

The same principle, one step closer: the browser keeps objects on the local disk and does not ask for them again.

The page is fixed on the server and still wrong on your screen

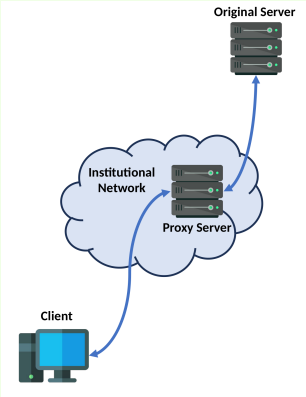
The browser is serving its own copy, and never asked. A normal reload may not be enough: it takes a forced reload, or emptying the cache, to make it go and look.

Every web developer meets this on the first day.



Beyond caching: the institutional proxy

- A proxy is not only about speed: it is how an institution grants access to services of its own.
- Requests forwarded by `proxy.unina.it` reach the origin server as requests from the university — which is what a subscription is sold against.
- The origin server sees the proxy's address, and reads the real destination off the `Host:` header line.

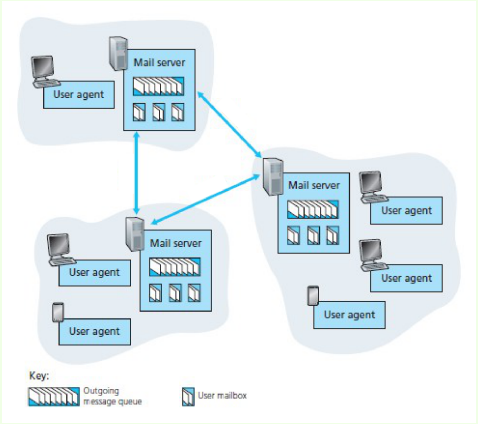


Commercial and free proxies exist for the same reason, used the other way round.

Electronic mail: three components

- User agent: the program that composes and reads — Thunderbird, K-9 Mail, a webmail page.
- Mail server: always on, holds one mailbox per user and an outgoing queue.
- SMTP: the protocol that moves a message from one mail server to another.

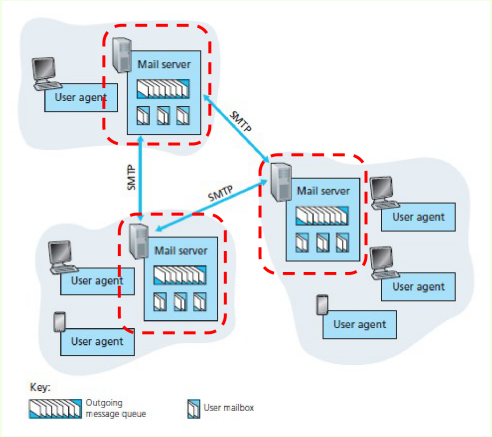
Unlike the Web, the communication also happens between servers: two protocols are needed, not one.



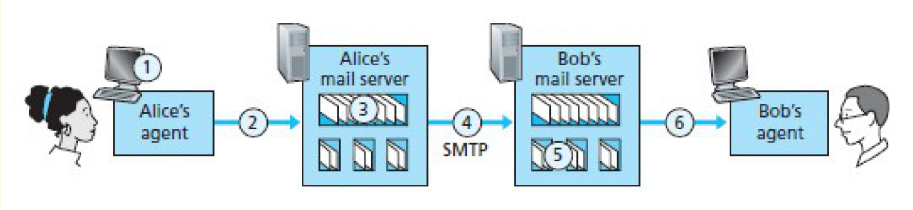
SMTP runs between mail servers

- Users do not exchange mail directly: a specialised, always-on server sits at each end.
- Every mail server runs both sides of SMTP — client when it sends, server when it receives.
- It uses TCP on port 25, and relies on it entirely for getting the bytes across intact.

A user agent hands its message to its own server, in SMTP or in HTTP. The red legs are the ones SMTP was made for.



Alice sends Bob a message



1. Alice writes the message and gives her agent Bob's address.
2. The agent delivers it to her own mail server, which queues it.
3. That server's SMTP client opens a TCP connection to Bob's mail server.
4. After the handshaking, it sends the message into the connection.
5. Bob's server receives it and drops it in Bob's mailbox.
6. Bob reads it whenever he likes.

Why the message goes through Alice's server

Why not send it straight from Alice's machine to Bob's server?

- Because Bob's server may be down, and Alice's laptop is not going to sit there retrying for three days.
- Her mail server will: it keeps the message in the outgoing queue and tries again every half hour or so.
- After several days without success it gives up and tells Alice — by mail.
- Servers are always on, always reachable, and their identity is certified: three things an end system is not.

There are no intermediate mail servers, though: Hong Kong to St. Louis is one direct TCP connection.

An SMTP dialogue

Client `crepes.fr`, server `hamburger.edu`, TCP already established.

```
S: 220 hamburger.edu
C: HELO crepes.fr                    today EHLO: the extended greeting
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr ... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: .                                 a lone period ends the message
S: 250 Message accepted for delivery
C: QUIT
```

Five commands, and a numeric reply to each one — the same idea as an HTTP status code. The connection is persistent: more messages follow here.

The message itself, and its 7-bit past

Inside `DATA` travels a message with a header of its own, defined by a different standard:

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Searching for the meaning of life.

(the body, after the blank line)
```

- These are not the SMTP commands, even where the words coincide: the commands drive the delivery, these travel inside it.
- SMTP is older than HTTP and restricts the whole message, body included, to 7-bit ASCII.
- So an image is encoded into letters before it leaves and decoded on arrival — which is why an attachment is bigger than the file.

Pull, not push

Alice's message is in Bob's mailbox, on a machine that is not his. He still has to get it.

- SMTP cannot do it: it is a push protocol, it delivers where it is told. Fetching is a pull operation.
- Putting the mail server on Bob's laptop would work, and would require the laptop to be on and reachable at all times.
- So a second protocol is needed, between the user agent and the mail server — and the server authenticates Bob before handing anything over.

In webmail that second protocol is HTTP and the user agent is the browser. Between the servers it is still SMTP: nothing changed there.

POP3 and IMAP

POP3: the old and simple one, on port 110. Three phases over one TCP connection:

1. authorization: username and password;
2. transaction: list, retrieve, mark for deletion;
3. update: on `quit`, the marked messages are removed.

Messages are downloaded and deleted. Folders and searching are your own machine's problem.

IMAP: what mail clients actually use today, together with HTTP for webmail:

- folders on the server, created and managed remotely;
- search inside those folders, without downloading;
- fetch part of a message — headers only, one attachment;
- several clients on the same mailbox at once.

Your phone and your laptop show the same folders because the folders are not on either of them.

Summary

- An HTTP message is text: a first line, header lines, a blank line, and an optional body.
- The first line carries a method on the way out and a status code on the way back.
- Cookies build a session on top of a stateless protocol, out of two header lines and two stores.
- A cache answers on behalf of the origin server, and pays for itself on the access link rather than on the clock.
- The conditional GET keeps a cached copy honest with two small messages instead of the object.
- Mail needs two protocols: SMTP pushes between servers, IMAP or HTTP pulls the mailbox to the user.

Why you will not see plain HTTP any more

Type `http://` into a browser today and it is very unlikely that plain HTTP ever leaves your machine.

- HTTPS: the same HTTP, over TLS, on port 443. The messages are identical; nobody in between can read them.
- HSTS: a header line in which a site declares that browsers must never use plain HTTP for it again, for months.
- Browsers ship a preload list of such sites and an HTTPS-first mode, so the URL is rewritten before any packet is sent.

So what does the `http` display filter show?

Nothing. The traffic is there — TCP to port 443 — but Wireshark sees TLS records, not requests. What TLS does inside them is a question for the end of the course.

A response that does not fit in one packet

A page long enough that its response must be split across several TCP segments.

1. Empty the browser cache, so that nothing is served locally: in Firefox or Chrome, `Ctrl-Shift-Del`, or disable it in the developer tools.
2. Start the Wireshark capture, with no display filter yet.
3. Fetch the page with a client that will not upgrade the request:

```
$ curl http://info.cern.ch/hypertext/WWW/TheProject.html > /dev/null
```

4. Stop the capture, then type `http` in the display filter to find the two HTTP messages.
5. Remove the filter again: the response you are looking for is mostly in packets that are not HTTP messages at all.

Questions

(a) How many HTTP `GET` messages did your client send, and which packet number carries the one for the page?

(b) Which packet number carries the status code and phrase of the response?

(c) What are that status code and phrase?

(d) How many TCP segments carrying data were needed for that single response? Look for TCP segment of a reassembled PDU in the Info column.

Then repeat the whole thing with a browser instead of `curl`, and count the HTTP messages again. Where did the page go?